Implementation of Transformations, Viewing, and Projections

HARSH PRITEHSKUMAR MISTRY, Indraprastha Institute of Information Technology, India

This project focuses on implementing transformations, viewing, and projections on a 3D cube using C++ and OpenGL, with GLM for matrix operations and ImGui for user interaction. The project includes deriving a 4x4 matrix for rotating about an arbitrary axis using a change of basis, creating a custom `lookAt` function, and generating different perspective views.

ACM Reference Format:
Harsh Pritehskumar Mistry. 2024. Implementation of Transformations, Viewing, and Projections. 1, 1 (September 2024), 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The aim of this project is to implement various transformations and viewing techniques on a 3D cube using C++. The project demonstrates manual matrix construction for rotation about an arbitrary axis, and the development of a custom `lookAt` function to simulate camera views. Additionally, perspective transformations are applied to visualize different projections of the cube, including one-point, two-point, and three-point perspectives.

2 QUESTION 1: ROTATION MATRIX DERIVATION AND IMPLEMENTATION

2.1 Deriving the 4x4 Rotation Matrix

To rotate a 3D object 30 degrees counterclockwise about an arbitrary axis, we first normalize the axis of rotation $(1, 2, 2)$ and construct a rotation matrix using the change of basis. The matrix is derived as follows:

(1) Normalize the axis vector.
(2) Compute two perpendicular vectors v_1 and v_2 using the cross product.
(3) Construct the matrix C using v_1, v_2, and the axis vector.
(4) The inverse of C, denoted C^{-1}, is the transpose of C.
(5) Perform the rotation in the local coordinate system using a standard rotation matrix R_z about the z-axis.
(6) The final transformation is $M = C^{-1} \cdot R_z \cdot C$.

2.2 Code Implementation

The rotation matrix is implemented in the `createRotationMatrix()` function, as shown below:

```
glm::mat4 createRotationMatrix() {
    glm::vec3 axis = glm::normalize(glm::vec3(1.0f, 2.0f, 2.0f));
    glm::vec3 t = glm::vec3(3.0f, 1.0f, 2.0f);
    glm::vec3 v1 = glm::normalize(glm::cross(axis, t));
    glm::vec3 v2 = glm::cross(axis, v1);
    glm::mat4 C(1.0f);
```

Author's address: Harsh Pritehskumar Mistry, harsh22200@iiitd.ac.in, Indraprastha Institute of Information Technology, Delhi, Delhi, India.

ACM XXXX-XXXX/2024/9-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
    C[0][0] = v1.x; C[0][1] = v2.x; C[0][2] = axis.x;
    C[1][0] = v1.y; C[1][1] = v2.y; C[1][2] = axis.y;
    C[2][0] = v1.z; C[2][1] = v2.z; C[2][2] = axis.z;
    glm::mat4 C_inv = glm::transpose(C);
    float theta = glm::radians(30.0f);
    glm::mat4 R_z(1.0f);
    R_z[0][0] = cos(theta); R_z[1][0] = -sin(theta);
    R_z[0][1] = sin(theta); R_z[1][1] = cos(theta);
    return C_inv * R_z * C;
}
```

2.3 Normalize the Axis of Rotation $\mathbf{a}$

The axis of rotation is $\mathbf{a} = (1, 2, 2)$.

To normalize $\mathbf{a}$:

$$\mathbf{a}_{\text{normalized}} = \frac{\mathbf{a}}{\|\mathbf{a}\|}$$

$$\|\mathbf{a}\| = \sqrt{1^2 + 2^2 + 2^2} = \sqrt{9} = 3$$

$$\mathbf{a}_{\text{normalized}} = \left(\frac{1}{3}, \frac{2}{3}, \frac{2}{3}\right)$$

Thus, the normalized axis of rotation is:

$$\mathbf{a}_{\text{normalized}} = \left(\frac{1}{3}, \frac{2}{3}, \frac{2}{3}\right)$$

2.4 Compute the First Perpendicular Vector $\mathbf{v}_1$

Next, we compute the vector $\mathbf{v}_1$ as the cross product of $\mathbf{a}_{\text{normalized}}$ and $\mathbf{t} = (3, 1, 2)$.

$$\mathbf{v}_1 = \mathbf{a}_{\text{normalized}} \times \mathbf{t}$$

Using the cross product formula:

$$\mathbf{v}_1 = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ \frac{1}{3} & \frac{2}{3} & \frac{2}{3} \\ 3 & 1 & 2 \end{vmatrix}$$

$$\mathbf{v}_1 = \hat{i}\left(\frac{2}{3} \cdot 2 - \frac{2}{3} \cdot 1\right) - \hat{j}\left(\frac{1}{3} \cdot 2 - \frac{2}{3} \cdot 3\right) + \hat{k}\left(\frac{1}{3} \cdot 1 - \frac{2}{3} \cdot 3\right)$$

$$\mathbf{v}_1 = \hat{i}\left(\frac{4}{3} - \frac{2}{3}\right) - \hat{j}\left(\frac{2}{3} - 2\right) + \hat{k}\left(\frac{1}{3} - 2\right)$$

$$\mathbf{v}_1 = \hat{i}\left(\frac{2}{3}\right) - \hat{j}\left(-\frac{4}{3}\right) + \hat{k}\left(-\frac{5}{3}\right)$$

$$\mathbf{v}_1 = \left(\frac{2}{3}, \frac{4}{3}, -\frac{5}{3}\right)$$

Thus, $\mathbf{v}_1$ is:

$$\mathbf{v}_1 = \left(\frac{2}{3}, \frac{4}{3}, -\frac{5}{3}\right)$$

2.5 Normalize $\mathbf{v}_1$

Now, we normalize $\mathbf{v}_1$:

$$\|\mathbf{v}_1\| = \sqrt{\left(\frac{2}{3}\right)^2 + \left(\frac{4}{3}\right)^2 + \left(-\frac{5}{3}\right)^2}$$

$$\|\mathbf{v}_1\| = \sqrt{\frac{4}{9} + \frac{16}{9} + \frac{25}{9}} = \sqrt{\frac{45}{9}} = \sqrt{5}$$

$$\mathbf{v}_{1\text{normalized}} = \frac{1}{\sqrt{5}}\left(\frac{2}{3}, \frac{4}{3}, -\frac{5}{3}\right)$$

$$\mathbf{v}_{1\text{normalized}} = \left(\frac{2}{3\sqrt{5}}, \frac{4}{3\sqrt{5}}, -\frac{5}{3\sqrt{5}}\right)$$

2.6 Compute the Second Perpendicular Vector $\mathbf{v}_2$

Next, we compute $\mathbf{v}_2$, which is perpendicular to both $\mathbf{a}_{\text{normalized}}$ and $\mathbf{v}_1$. We compute $\mathbf{v}_2$ as the cross product of $\mathbf{a}_{\text{normalized}}$ and $\mathbf{v}_1$:

$$\mathbf{v}_2 = \mathbf{a}_{\text{normalized}} \times \mathbf{v}_1$$

2.7 Construct the Matrix C

Once we have $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{a}_{\text{normalized}}$, we can construct the matrix C as:

$$C = \begin{pmatrix} v_1.x & v_2.x & a_{\text{normalized}}.x & 0 \\ v_1.y & v_2.y & a_{\text{normalized}}.y & 0 \\ v_1.z & v_2.z & a_{\text{normalized}}.z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.8 Compute the Inverse of C

The inverse of C is simply the transpose of the upper-left 3x3 matrix, since it's an orthogonal matrix.

2.9 Perform the Rotation in Local Coordinates

The rotation about the z-axis in local coordinates is given by the matrix:

$$R_z = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where $\theta = 30^\circ$ or $\theta = \frac{\pi}{6}$.

h. Compute the Final Rotation Matrix

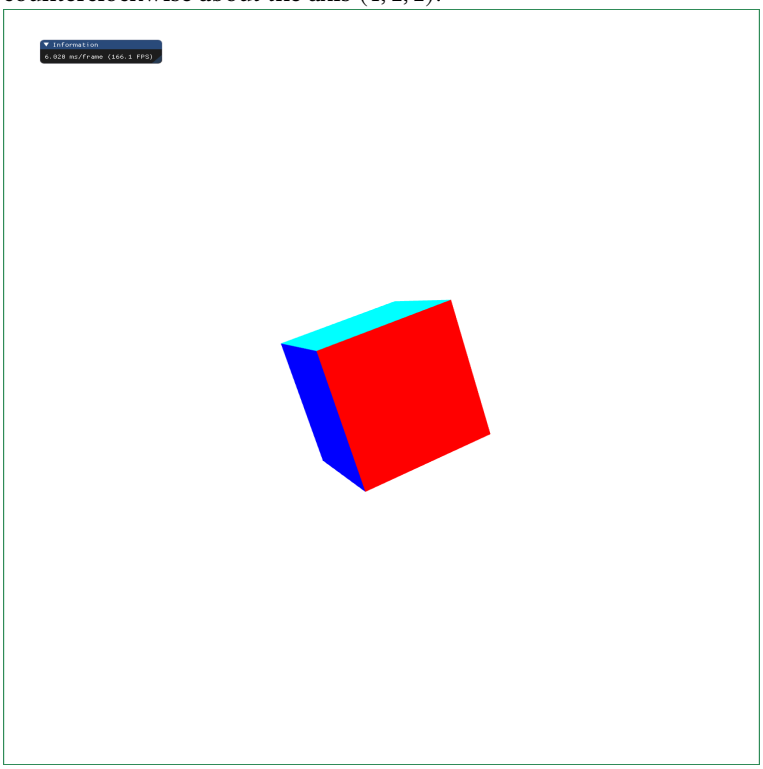
The final rotation matrix is given by:

$$M = C^{-1} \cdot R_z \cdot C$$

This matrix will perform a 30-degree rotation about the arbitrary axis $(1, 2, 2)$.

3 QUESTION 2: SCREENSHOT AFTER APPLYING ROTATION

A screenshot is generated after applying the above rotation matrix to the cube. The cube is rotated by 30 degrees counterclockwise about the axis $(1, 2, 2)$.



4 QUESTION 3: CUSTOM LOOKAT FUNCTION

4.1 Custom LookAt Function Implementation

GLM provides a `lookAt()` function for viewing transformations, which constructs the view matrix. The custom `lookAt` function calculates the view matrix using the eye position, the gaze direction, and the up vector. The steps are as follows:

(1) Calculate the w vector (gaze direction).
(2) Calculate the u vector (right direction) using the cross product of the up vector and w.
(3) Compute the v vector (up direction).
(4) Construct the view matrix based on these vectors.

The code for the custom `lookAt` function is:

```
glm::mat4 customLookAt(const glm::vec3& eye, const glm::vec3& center, const glm::vec3& up) {
    glm::vec3 w = glm::normalize(eye - center);
    glm::vec3 u = glm::normalize(glm::cross(up, w));
    glm::vec3 v = glm::cross(w, u);
    glm::mat4 lookAtMatrix(1.0f);
    lookAtMatrix[0][0] = u.x; lookAtMatrix[1][0] = u.y; lookAtMatrix[2][0] = u.z;
```

```
    lookAtMatrix[0][1] = v.x; lookAtMatrix[1][1] = v.y; lookAtMatrix[2][1] = v.z;
    lookAtMatrix[0][2] = w.x; lookAtMatrix[1][2] = w.y; lookAtMatrix[2][2] = w.z;
    return lookAtMatrix;
}
```

4.2 Rendering the Rotated Cube

The rotated cube is rendered using the custom `lookAt` function with the following camera parameters:

- Position: $(50, 100, 20)$
- Gaze: center of the rotated cube
- Up: $(0, 0, 1)$

4.3 Verification Against GLM's LookAt Function

The custom `lookAt` function is verified by comparing its output with GLM's built-in `lookAt()` function.

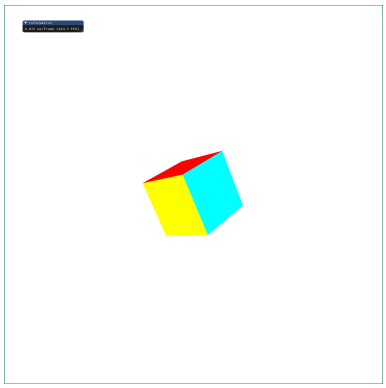


Fig. 1. Custom `lookAt` function output

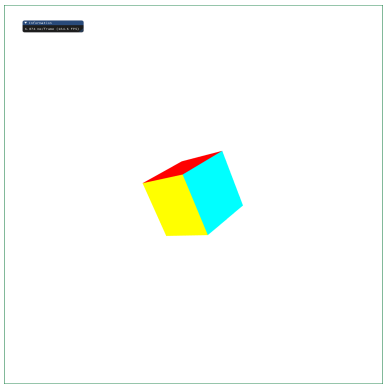


Fig. 2. `glm::lookAt()` function output

5 QUESTION 4: PERSPECTIVE VIEWS OF THE CUBE

5.1 Different Perspectives

The following viewing transformations are applied to the original (unrotated) cube:

(1) One-point perspective
(2) Two-point perspective
(3) Three-point (bird's eye) perspective
(4) Three-point (rat's eye) perspective

The viewing matrices are constructed using the custom `lookAt` function with different camera parameters.

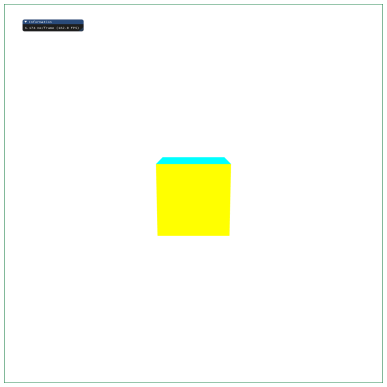


Fig. 3. One-point perspective

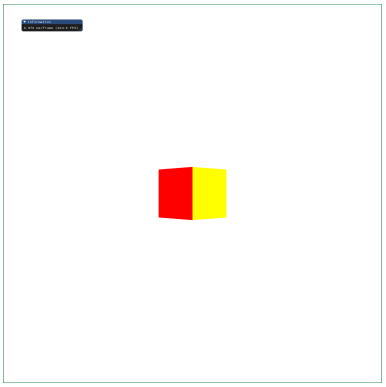


Fig. 4. Two-point perspective

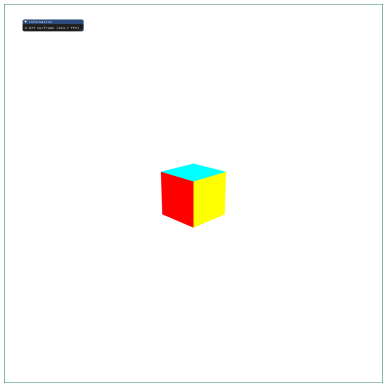


Fig. 5. Three-point (bird's eye) perspective

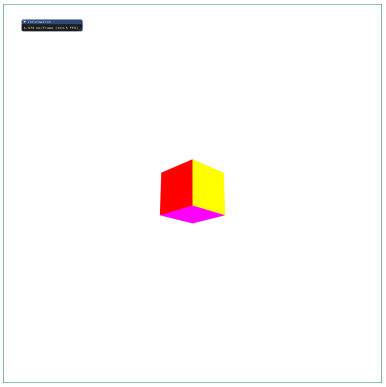


Fig. 6. Three-point (rat's eye) perspective

6 CONCLUSION

The project successfully demonstrates the implementation of 3D transformations and viewing techniques using C++. It includes the derivation and implementation of a rotation matrix, a custom `lookAt` function, and various perspective views of a 3D cube. Future improvements could include adding more advanced interaction methods or integrating animation techniques.